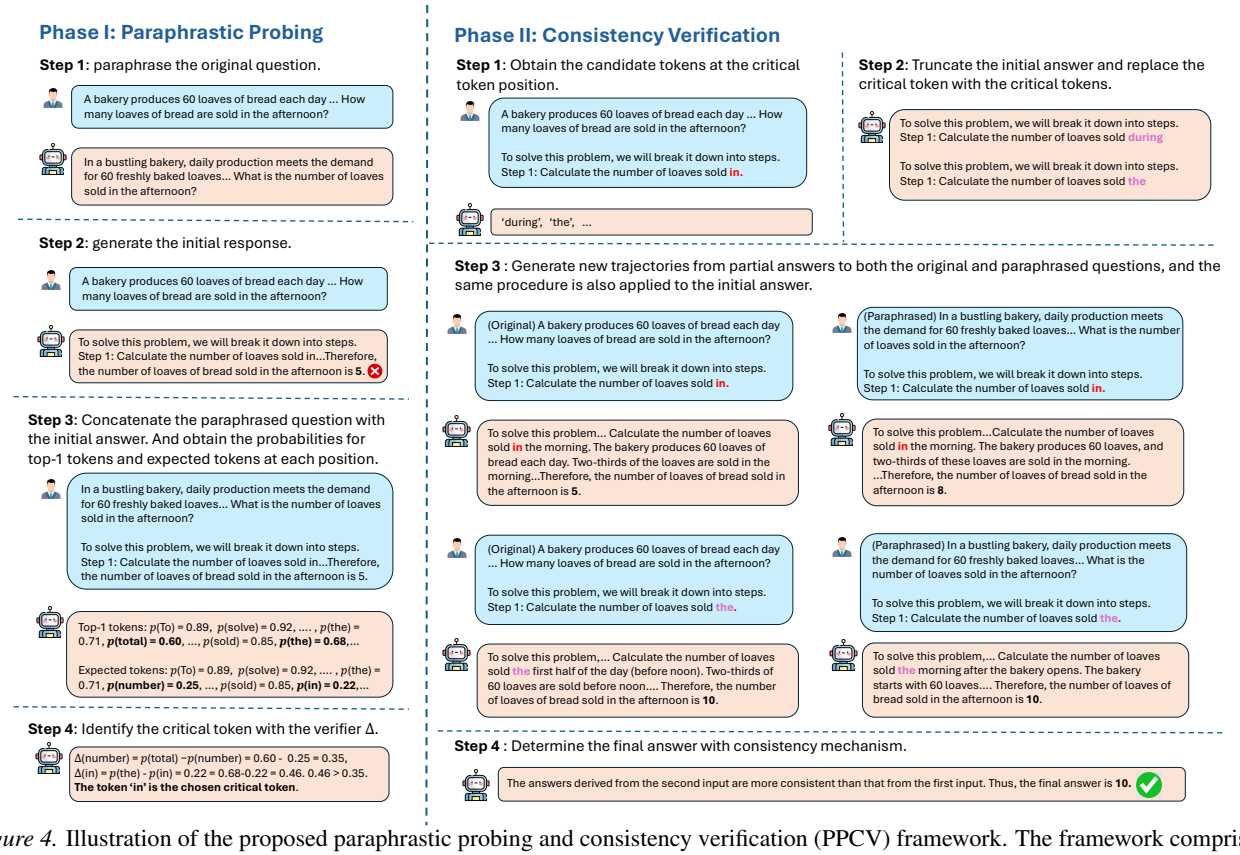
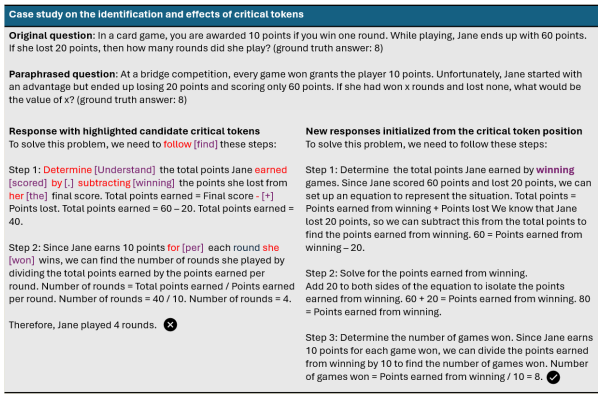


Figure 4. Illustration of the proposed paraphrastic probing and consistency verification (PPCV) framework. The framework comprises two stages: (i) probing critical tokens through paraphrased forms, and (ii) rolling out new reasoning steps with alternative tokens and selecting the final answer using the paraphrase consistency verification mechanism.



Case study on the identification and effects of critical tokens

Original question: In a card game, you are awarded 10 points if you win one round. While playing, Jane ends up with 60 points. If she lost 20 points, then how many rounds did she play? (ground truth answer: 8)

Paraphrased question: At a bridge competition, every game won grants the player 10 points. Unfortunately, Jane started with an advantage but ended up losing 20 points and scoring only 60 points. If she had won x rounds and lost none, what would be the value of x? (ground truth answer: 8)

Response with highlighted candidate critical tokens
To solve this problem, we need to follow [find] these steps:

Step 1: Determine [Understand] the total points Jane earned [scored] by [.] subtracting [winning] the points she lost from her [the] final score. Total points earned = Final score - [+] Points lost. Total points earned = 60 – 20. Total points earned = 40.

Step 2: Since Jane earns 10 points for [per] each round she [won] wins, we can find the number of rounds she played by dividing the total points earned by the points earned per round. Number of rounds = Total points earned / Points earned per round. Number of rounds = 40 / 10. Number of rounds = 4.

Therefore, Jane played 4 rounds.

New responses initialized from the critical token position
To solve this problem, we need to follow these steps:

Step 1: Determine the total points Jane earned by winning games. Since Jane scored 60 points and lost 20 points, we can set up an equation to represent the situation. Total points = Points earned from winning + Points lost We know that Jane lost 20 points, so we can subtract this from the total points to find the points earned from winning. 60 = Points earned from winning – 20.

Step 2: Solve for the points earned from winning. Add 20 to both sides of the equation to isolate the points earned from winning. 60 + 20 = Points earned from winning. 80 = Points earned from winning.

Step 3: Determine the number of games won. Since Jane earns 10 points for each game won, we can divide the points earned from winning by 10 to find the number of games won. Number of games won = Points earned from winning / 10 = 8.

Figure 5. Case study illustrating the identification and effects of critical tokens identified by our method. Tokens highlighted in red indicate candidate critical tokens, whereas tokens highlighted in purple correspond to alternative tokens generated when conditioning on paraphrased questions.

original form, highlighting the potential of paraphrasing to uncover pivotal tokens that are critical for successful reasoning. Motivated by this observation, we introduce paraphrastic probing to efficiently identify the critical token. Given the original question q_0, we first prompt the LLM to generate multiple paraphrased forms, denoted as $q_1, q_2, ..., q_N$, where N is the number of the paraphrased questions. We adopt Automatic Prompt Engineering (APE) (Zhou et al., 2022) to derive paraphrasing instructions that preserve semantic integrity and all numerical values, mathematical relationships, and core logical structures of the problem, while maximizing linguistic and contextual diversity. Additional details can be found in Appendix B. We then obtain the initial reasoning path $r_0^{q_0}$ for the original question using greedy decoding. This reasoning path is subsequently concatenated with each paraphrased question, and the resulting synthetic inputs are fed into the LLM to compute the token probability distribution at each position in $r_0^{q_0}$. Specifically, the token probability distribution at ith position conditioned on the paraphrased question q_n is expressed as

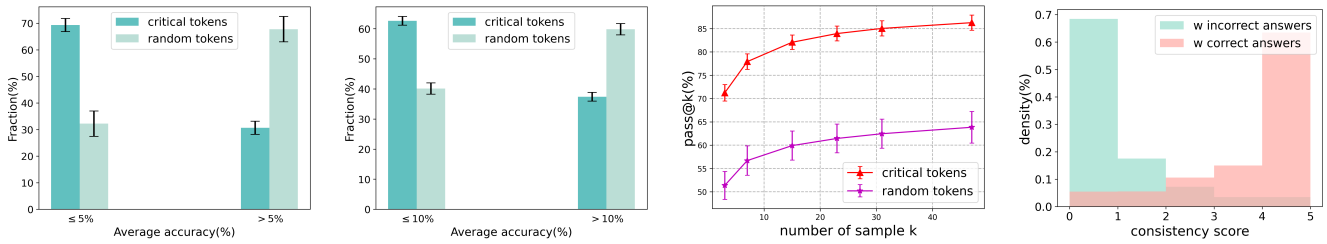
$$P_i^{q_n} = \text{LLM}(\tilde{a}_i | \mathcal{I}, q_n, r_{0,<i}^{q_0}), \tag{1}$$

where $\mathcal{I}$ denotes the instruction prefix and $\tilde{a}_i$ represents the sampled token at ith position. The token $\tilde{a}_i$ is regarded as a candidate critical token if predicted top-1 token does not match the expected token at the same position in $r_0^{q_0}$, i.e.,

$$\arg\max P_i^{q_n} \neq a_i, \tag{2}$$

where a_i denotes the token at the ith position in $r_0^{q_0}$.

To validate the effectiveness of our method in identifying critical tokens and to demonstrate the pivotal role these



(a) *(b)* *(c)* *(d)*

Figure 6. (a) Comparison of the fraction of tokens whose associated rollout correctness falls below or above 5%, for tokens extracted by our method versus randomly selected tokens. (b) Comparison of the fraction of the subsequent tokens whose associated rollout correctness falls below or above 10%, for tokens extracted by our method versus randomly selected tokens. (c) Comparison of the impact of extracted tokens by our method versus random tokens on LLM reasoning performance. (d) Comparison of the density distributions of consistency scores for rollouts with correct and incorrect answers.

tokens play when extracted via paraphrastic probing, we conduct a case study illustrated in Figure 5. Because large language models can adjust their reasoning trajectories under the paraphrased form of the question, certain tokens in the original reasoning sequence produce different subsequent tokens when conditioned on paraphrased inputs. In this example, most candidate critical tokens are located within incorrect reasoning segments. Notably, when we identify "subtracting" as a critical token and replace it with an alternative token (i.e., "winning"), the resulting rollout corrects the erroneous reasoning steps and yields the correct final answer. This example highlights the impact of critical tokens and underscores the effectiveness of our method in identifying tokens that are pivotal to reasoning outcomes.

In addition, we conduct a quantitative analysis to examine the authenticity and impact of tokens extracted via paraphrastic probing, comparing them against randomly selected tokens. Specifically, we sample 100 instances with incorrect reasoning trajectories from the GSM8K (Cobbe et al., 2021) training sets. Following the paraphrastic probing pipeline, we identify candidate critical tokens in the early reasoning steps. In each run, we randomly sample 40% of the candidate critical tokens for evaluation and repeat the experiment independently 10 times. For comparison, we apply the same procedure to randomly selected tokens. All evaluations are conducted using Llama-3.1-8B-Instruct (Grattafiori et al., 2024). We first compute the average accuracy of rollouts generated from partial trajectories truncated at the position of the extracted tokens, and compare these results with those obtained from random-token truncation. As shown in Figure 6a, a large proportion of the extracted tokens exhibit near-zero correctness, consistent with the first criterion of critical tokens. We further evaluate the average accuracy of rollouts initiated from subsequent tokens (5–10 tokens). The results in Figure 6b indicate that errors persist beyond the identified positions, supporting the second criterion that critical tokens induce sustained degradation in downstream reasoning. Finally, we substitute the extracted critical tokens with alternative tokens and roll out multiple new reasoning trajectories. As shown in Figure 6c, replacing critical tokens leads to a significantly larger improvement in reasoning accuracy compared to substituting random tokens. These results further validate both the pivotal role of the identified critical tokens and the effectiveness of our paraphrastic probing method. Additional results can be found in Appendix A.

We introduce a heuristic verifier to select the final critical token from multiple candidates. For a candidate token a_i and paraphrased question q_n, the verification score is defined as

$$\Delta_{q_n}(a_i) = \max P_i^{q_n} - P_i^{q_n}(\tilde{a}_i = a_i). \tag{3}$$

where $P_i^{q_n}$ denotes the predictive distribution at position i on question q_n. Intuitively, Δ measures how much the predicted top-1 token deviates from the expected token, indicating the token's potential impact on the reasoning trajectory. For each extracted token a_i,we take the maximum score across paraphrases,

$$\Delta(a_i) = \max_{q_n} \Delta_{q_n}(a_i), \tag{4}$$

and select the final critical token as

$$a_c = \arg\max_i \Delta(a_i). \tag{5}$$

3.2. Consistency Verification

After identifying the final critical token a_c, we aim to refine the original reasoning path with alternative tokens and achieve final answer with paraphrase consistency mechanism. Specifically, we generate a set of alternative tokens $a_c^0, a_c^1, a_c^2, ..., a_c^{K-1}$ using the LLM conditioned on original question q_0, where a_c^0 is the original token in $r_0^{q_0}$ and the remaining tokens are sampled via top-K sampling. The initial reasoning path is truncated at the position of critical token, and each alternative token is concatenated to form synthetic inputs $\tilde{r}_c^0, \tilde{r}_c^1, \tilde{r}_c^2, ..., \tilde{r}_c^{K-1}$. We then roll out new reasoning trajectories for each synthetic input with respect to both the original and paraphrased questions using greedy decoding, denoted as $r_k^{q_0}, r_k^{q_1}, ..., r_k^{q_N}$ for $k = 0, 1, 2, ..., K-1$. Next, for the rollout with the kth alternative token, we compare

the answers obtained from the paraphrased forms with that from the original form and compute a consistency score $c_k = \sum_{n-1}^{N} \mathbb{I}(\Phi(r_k^{q_0}) = \Phi(r_k^{q_n}))$, where $\Phi(\cdot)$ and $\mathbb{I}(\cdot)$ denotes the function that extracts the final answer from a reasoning trajectory and the indicator function, respectively. The answer associated with the highest consistency score is then selected as the final prediction

$$\text{ans}_f = \Phi(r_k^{q_0}), \text{where } k = \arg\max_k c_k. \tag{6}$$

To justify our paraphrase consistency mechanism, we investigate the impact of paraphrased forms on LLM reasoning. We sample instances from the GSM8K (Cobbe et al., 2021) and follow our pipeline to extract critical tokens. From each truncated reasoning trajectory, we roll out multiple reasoning paths by concatenating alternative tokens. For each original question, we generate five paraphrased variants and compute the consistency score for resulting rollouts. The evaluation is conducted on Llama-3.1-8B-Instruct (Grattafiori et al., 2024). We then analyze the distribution of consistency scores for rollouts that yield correct versus incorrect answers. As shown in Figure 6d, more than 90% of rollouts with correct answers achieve a consistency score of at least 1, whereas this proportion drops to around 30% for rollouts with incorrect answers. This sharp contrast indicates that correct rollouts are more robust across paraphrased variants, motivating the design of our paraphrase consistency mechanism to exploit this property for improved final predictions.

To address potential collisions when multiple answers obtain the same maximum consistency score, we introduce similarity-weighted consistency verification. Inspired by weighted majority voting (Dogan & Birant, 2019), this approach adjusts the influence of each paraphrased form on the consistency score according to its similarity to the original form. Intuitively, paraphrased forms with lower similarity should exert greater weight, as they provide stronger evidence of robustness, whereas those closely resembling the original form contribute less. Concretely, we first extract embeddings for both the original and paraphrased questions and compute their similarity scores as $s_n = \text{sim}(q_0, q_n)$, where $\text{sim}(\cdot)$ denotes a similarity measure such as cosine similarity. We then derive weights via a softmax function $w_n = \text{softmax}(s_n) = \frac{\exp(-\lambda s_n)}{\sum_n \exp(-\lambda n)}$, where λ is the temperature scaling coefficient. Finally, the similarity-weighted consistency score is defined as $\tilde{c}_k = \sum_{n-1}^{N} w_n \mathbb{I}(\Phi(r_k^{q_0}) = \Phi(r_k^{q_n}))$. This ensures agreement with more diverse paraphrases contributes more strongly to the final decision.

3.3. Discussion

Our technical contributions differ from prior works in three distinct ways. First, prior works (Zhou et al., 2024; Chen et al., 2024b; Yadav et al., 2024) typically use paraphrasing merely to expand the solution space. In contrast, we introduce Paraphrastic Probing, a mechanism that uses paraphrasing to test the model's internal confidence. By analyzing the discrepancy in token-level logits of the initial trajectory between the original and paraphrased questions, we can rigorously pinpoint the critical tokens that may lead to errors in the following steps.This transforms paraphrasing from a generation tool into a precise, token-level diagnostic tool. Second, prior works (Zhou et al., 2024; Chen et al., 2024b) typically rely on simple majority voting across multiple solutions. Our Paraphrase Consistency mechanism is technically distinct. It validates answers based on their robustness across semantic variations of the problem constraint. We further introduce a similarity-weighted consistency metric that weighs answers based on the linguistic diversity of the paraphrase, offering a more nuanced selection criterion than simple frequency counts. At last, a major technical limitation in current reasoning research is the reliance on external models or human-annotated error steps. Our method contributes a fully self-contained pipeline that identifies and corrects errors using the model's own sensitivity to surface-form perturbations. More discussion on the impact of critical tokens on correct trajectory can be found in Appendix D.

Besides, although we select the top candidate for the primary experiments to maintain computational efficiency, the framework itself naturally extends to the multi–critical-token setting. For multiple critical tokens, we can generate alternative tokens for each identified position and apply paraphrase consistency across the new rollouts. This allows the model to refine multiple segments of its intermediate reasoning steps rather than only one. The details of the algorithm can be found in Appendix C.

Table 1. Comparison of our method with baseline approaches on Llama-3.1-8B-Instruct and Mistral-7B-Instruct-v0.2.

Model	Method	GSM8K	GSM-Hard	Math500	SVAMP	ARC
Llama-3.1	Chain-of-Thought	77.40	28.00	31.00	83.00	58.91
	Self-Consistency	80.60	31.80	37.80	85.10	60.75
	Tree-of-Thought	75.74	33.28	31.60	81.20	80.72
	Guided Decoding	75.51	32.45	31.20	81.70	81.74
	Predictive Decoding	81.43	40.26	34.00	85.90	84.56
	Phi-Decoding	86.58	39.88	38.20	84.50	85.41
	PPCV (Ours)	**88.24**	**49.73**	**50.00**	**89.60**	**88.31**
Mistral-7B	Chain-of-Thought	46.45	26.91	12.20	62.40	41.42
	Self-Consistency	50.38	28.65	14.20	66.70	44.54
	Tree-of-Thought	50.49	25.78	11.40	60.60	41.04
	Guided Decoding	50.79	27.07	14.00	62.90	39.51
	Predictive Decoding	55.67	27.07	14.40	62.10	47.87
	Phi-Decoding	**56.60**	28.43	13.40	63.20	60.24
	PPCV (Ours)	56.58	**31.08**	**14.60**	**69.30**	**69.88**

Table 2. Comparison of our method with baseline approaches on Qwen3-32B (non-thinking mode).

Model	Method	AIME2024	AIME2025	BRUMO2025	HMMT2025
Qwen3-32B	Chain-of-Thought	30.00	23.67	30.00	9.67
	Guided Decoding	26.67	22.67	28.67	7.33
	Predictive Decoding	32.67	24.00	33.33	10.33
	Phi-Decoding	33.60	24.33	36.67	10.67
	PPCV (Ours)	**40.00**	**26.00**	**43.33**	**13.33**